
CS5340 Assignment 1:

Belief Propagation and Maximal Probability

1 Overview

Welcome to the course! In this first assignment, you will write code to perform inference on simple tree-like graphs using belief propagation. You will write the code for both the sum-product and max-product algorithms.

References: Lectures 4 and 5

Honour Code. This coding assignment constitutes **15%** of your final grade in CS5340. Note that plagiarism will not be condoned! You may discuss with your classmates and check the internet for references, but you **must not** submit code or a report copied directly from other sources.

2 Submission Instructions

Items to be submitted:

- **Source code (lab1.py).** This is where you fill in all your code.
- **Report (report.pdf).** This should describe your implementation and be no more than one page.

Please indicate clearly your name and student number (the one that looks like A1234567X) in the report as well as at the top of your source code. Zip the two files together and name the archive in the following format: A1234567X_lab1.zip (replace this with your student number).

Submit your assignment by **13 September 2026** to Canvas. 25% of the total score will be deducted for each day of late submission.

3 Getting Started

This assignment, as well as the subsequent ones, requires Python 3.5 or later. The required Python packages can be installed using:

```
pip install -r requirements.txt
```

If you have any issues with the installation, please post them in the forum so that other students or the instructors can help accordingly.

4 Test Cases

To help with your implementation, we have provided sample datasets and their expected outputs. They can be found in the `data/` folder. The code that loads the sample data and checks your program output is in `test_lab1.py`.

We will mainly work with the Bayesian network stored in `graph_small.json`, shown in Figure 1. We also provide a larger graphical model in `graph_large.json` for further testing.

During grading, we will run your code on hidden test cases in addition to the two provided graphs.

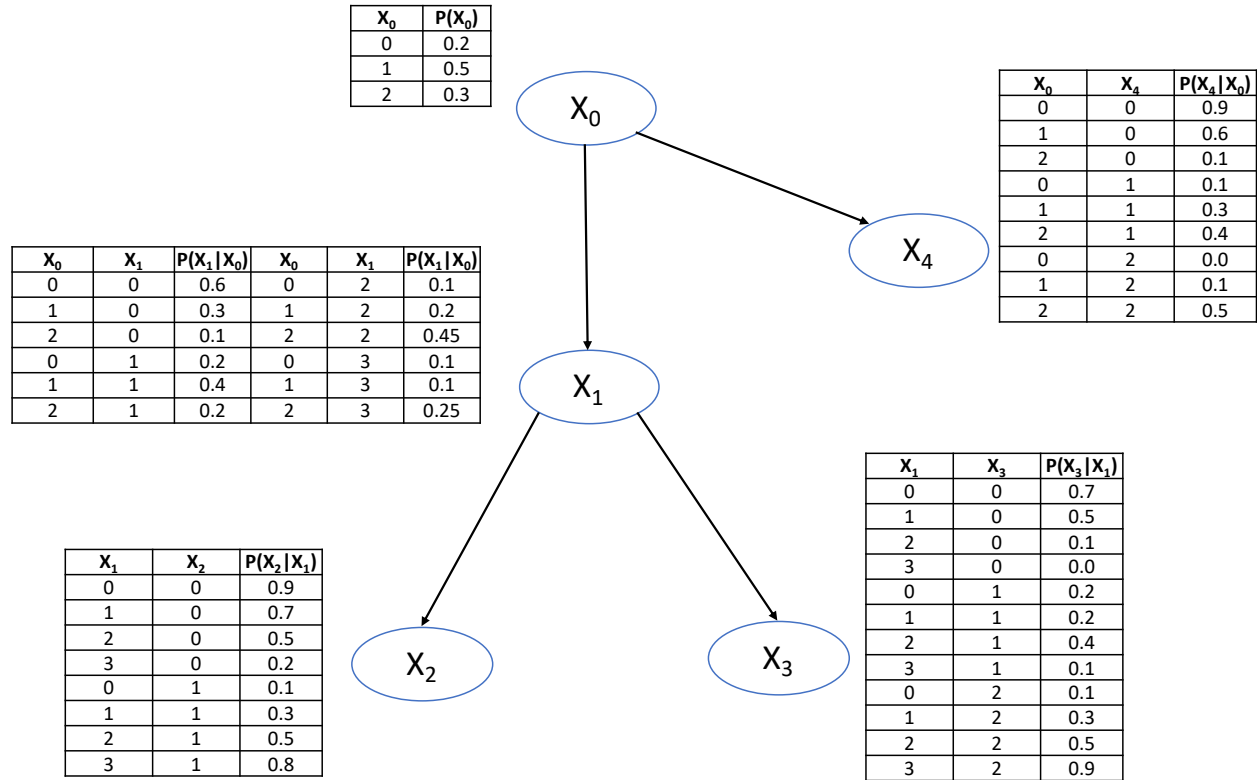


Figure 1: The Bayesian network you will perform inference on.

5 Basic Factor Operations

As you recall from the lectures, we use factor tables to represent the conditional probability distributions of a Bayesian network. You will now implement the core functionality for manipulating factor tables.

- **factor_product():** Compute the product of two factors.
- **factor_marginalize():** Sum over the indicated variable or variables and return the resulting factor.
- **observe_evidence():** Take a list of factors and the observed values of some variables, and set assignments inconsistent with those observations to zero.

All factor operations use a custom `Factor` class. Its implementation can be found in `factor.py`; a description of the datatype and example code can be found in `factor_readme.py`.

6 Naïve Summation

To motivate the need for belief propagation, we will first implement the naïve summation algorithm. That is, you will compute the required conditional probabilities by constructing the full joint distribution and then marginalising out irrelevant variables.

6.1 Computing the Joint Distribution

Complete `compute_joint_distribution()`, which computes the joint distribution over a Bayesian network. For a Bayesian network over variables $X_1, \dots, X_n$, the joint distribution is

$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Parent}(X_i)). \quad (1)$$

Generally, each node can have multiple parents, and the conditional probability should consider all parents. For the tree-like graphs in this assignment, however, nodes have at most one parent.

6.2 Computing Marginals

Having computed the joint distribution, we can obtain the marginal probabilities for a variable by marginalising out irrelevant variables. If we have observed evidence, we must also set assignments inconsistent with that evidence to zero. Complete `compute_marginals_naive()` by performing the following steps:

- Compute the joint distribution over all variables.
- Reduce the joint distribution using the evidence.
- Marginalise out irrelevant variables.

The resulting factor is unnormalised, so remember to normalise the final probability distribution.

For the small Bayesian network in Figure 1, a correct implementation gives the following marginal for $P(X_0 \mid X_3 = 1)$:

X_0	$P(X_0 \mid X_3 = 1)$
0	0.178
1	0.486
2	0.336

7 Sum-Product Algorithm

Even for the small graph in Figure 1, the joint distribution has

$$3 \times 4 \times 2 \times 3 \times 3 = 216$$

different assignments. This is large and does not scale. Standard variable elimination reduces this computational complexity but requires a separate run for every variable whose marginal we want to compute.

Belief propagation, or the sum-product algorithm, computes all single-node marginals for tree-like graphs in a single pass by reusing messages. It consists of the following phases:

- Apply evidence.
- Send messages inward towards the root.
- Send messages outward towards the leaves.
- Compute the marginal of each relevant variable.

The algorithm requires graph traversal. We use a [NetworkX](#) graph, and provide `generate_graph_from_factors()` to convert a list of factors into a NetworkX graph. For example, the neighbours of variable i can be accessed using

```
graph.neighbors(i)
```

Unary and pairwise factors are stored as node and edge attributes. The unary factor for variable i and pairwise factor between i and j can be accessed using

```
graph.nodes[i]['factor']
graph.edges[i, j]['factor']
```

If a node does not have a unary factor, it will not contain a `factor` field. You will have to handle the different scenarios yourself.

Complete `compute_marginals_bp()`, which computes marginal probabilities for multiple variables using belief propagation. If implemented correctly, its results should agree with `compute_marginals_naive()`.

Hint: It may be useful to create additional functions for this part. You may place those functions anywhere in the file.

8 MAP Inference Using Max-Product

In the final part, you will write code to compute the maximum a posteriori (MAP) configuration and its probability. To avoid underflow, we will work in log-space. Take the logarithm of all probabilities before message passing and perform all operations in log-space. Thus, max-product is implemented as max-sum.

First implement the relevant log-space factor operations:

- **factor_sum()**: Analogous to **factor_product()**, but for log-space.
- **factor_max_marginalize()**: Maximise over the indicated variable or variables and return the resulting factor. Keep track of the maximising values in **factor.val_argmax**.

Then complete **map_eliminate()**. It should take a list of factors and optional evidence, and compute the most probable joint assignment and its log probability.

For the small network, the MAP configuration conditioned on $X_3 = 1$ should be

$$\operatorname{argmax}_{x_0, x_1, x_2, x_4} P(X_0, X_1, X_2, X_4 \mid X_3 = 1) = \{X_0 = 0, X_1 = 0, X_2 = 0, X_4 = 0\}, \quad (2)$$

with unnormalised log probability

$$\log P(X_0 = 0, X_1 = 0, X_2 = 0, X_4 = 0, X_3 = 1) = -3.940. \quad (3)$$

Notice that $X_0 = 0$ in the MAP assignment, even though $X_0 = 1$ maximises the conditional marginal probability computed in the previous sections.